

SRI VENKATESHWARA COLLEGE OF ENGINEERING AND TECHNOLOGY

DEPARTMENT OF ELECTRONIC COMMUNICATION AND ENGINEERING

PROJECT REPORT

HOSPITAL MANAGEMENT SYSTEM

Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in Electronic Communication and Engineering

Guided by

Navven Prakash Sir

PROJECT TEAM

S.No.	Name	Roll Number
1	K. Prathibha	23781A0494
2	K. Lavanya	23781A0497
3	M. Harshitha	23781A04A6
4	M. Prasanna	23781A04A7
5	M. Tejaswini	23781A04B9
6	M. Anusha	23781A04C0
7	M. Keerthana	23781A04C2
8	M. Mythri	23781A04C4

IV B.Tech

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, **Navven Prakash Sir**, for his valuable guidance, encouragement and support throughout the development of this project. We also thank the Department of Electronic Communication and Engineering and Sri Venkateshwara College of Engineering and Technology for providing the facilities and academic support required to complete the project.

We are grateful to our faculty members, friends and family for their encouragement and assistance during the preparation of this project report.

ABSTRACT

The **Hospital Management System** is a menu-driven C language application designed to maintain basic hospital records in a simple and organized way. The system allows the user to add patient details, display patient records, search for a patient, update patient information, and delete records. It demonstrates the practical use of structures, arrays, functions, loops, conditional statements and menu-driven programming in C.

The main purpose of the project is to reduce manual record handling and provide a simple computerized method for managing patient information. The application is suitable as an academic project for understanding data management concepts and modular programming in C.

Keywords: Hospital Management, C Programming, Structures, Arrays, Functions, Patient Records, Menu-driven Application.

1. INTRODUCTION

Hospitals handle a large amount of information related to patients, doctors, appointments and treatments. Maintaining these records manually can be time-consuming and may lead to errors. A computerized management system provides a structured way to store and retrieve information.

This project implements a basic Hospital Management System using the C programming language. It focuses on patient record management and provides a simple menu through which the user can perform different operations.

1.1 Objectives

- To create a simple computerized system for maintaining patient records.
- To provide menu-based operations for adding, displaying, searching, updating and deleting records.
- To demonstrate the use of structures and functions in C.
- To reduce repeated manual work and improve record organization.
- To develop practical programming and problem-solving skills.

1.2 Scope

The current system is intended for academic demonstration and basic record management. It can be extended in the future with doctor management, appointment scheduling, billing, medicine records, file/database storage and user authentication.

2. SYSTEM MODULES

1. **Add Patient:** Accepts patient ID, name, age, gender, disease and contact number.
2. **Display Patients:** Displays all stored patient records in tabular form.
3. **Search Patient:** Searches for a patient using the patient ID.
4. **Update Patient:** Modifies the details of an existing patient.
5. **Delete Patient:** Removes a patient record from the current list.
6. **Exit:** Terminates the application.

2.1 Software Requirements

- Programming Language: C
- Compiler: GCC / Turbo C / Code::Blocks / Visual Studio Code with C compiler
- Operating System: Windows or Linux

2.2 Hardware Requirements

- Computer or laptop
- Minimum 2 GB RAM
- Keyboard and monitor
- Basic storage space

3. ALGORITHM / WORKING

Step 1: Start the program.

Step 2: Initialize the patient record array and patient count.

Step 3: Display the main menu.

Step 4: Read the user's choice.

Step 5: Perform the selected operation: Add, Display, Search, Update or Delete.

Step 6: After completing the operation, return to the main menu.

Step 7: If the user selects Exit, stop the program.

Step 8: End.

4. C PROGRAM

```
#include <stdio.h>
#include <string.h>

#define MAX 100

struct Patient {
    int id;
    char name[50];
    int age;
    char gender[10];
    char disease[50];
    char phone[15];
};

struct Patient p[MAX];
int count = 0;

void addPatient() {
    if (count >= MAX) {
        printf("\nPatient limit reached!\n");
        return;
    }

    printf("\nEnter Patient ID: ");
    scanf("%d", &p[count].id);

    printf("Enter Name: ");
    scanf(" %[^\\n]", p[count].name);

    printf("Enter Age: ");
    scanf("%d", &p[count].age);

    printf("Enter Gender: ");
    scanf("%s", p[count].gender);

    printf("Enter Disease: ");
    scanf(" %[^\\n]", p[count].disease);

    printf("Enter Phone Number: ");
    scanf("%s", p[count].phone);

    count++;
    printf("\nPatient added successfully!\n");
}

void displayPatients() {
    int i;

    if (count == 0) {
        printf("\nNo patient records found.\n");
        return;
    }

    printf("\n----- PATIENT RECORDS -----\\n");
    printf("ID\\tName\\tAge\\tGender\\tDisease\\t\\tPhone\\n");
    printf("-----\\n");

    for (i = 0; i < count; i++) {
        printf("%d\\t%-15s\\t%d\\t%-7s\\t%-15s\\t%s\\n",
            p[i].id, p[i].name, p[i].age,
            p[i].gender, p[i].disease, p[i].phone);
    }
}

int findPatient(int id) {
    int i;
    for (i = 0; i < count; i++) {
        if (p[i].id == id)
            return i;
    }
    return -1;
}

void searchPatient() {
    int id, pos;

    printf("\nEnter Patient ID to search: ");
```

```

scanf("%d", &id);

pos = findPatient(id);

if (pos == -1) {
    printf("\nPatient not found.\n");
} else {
    printf("\nPatient Found\n");
    printf("ID      : %d\n", p[pos].id);
    printf("Name     : %s\n", p[pos].name);
    printf("Age      : %d\n", p[pos].age);
    printf("Gender   : %s\n", p[pos].gender);
    printf("Disease  : %s\n", p[pos].disease);
    printf("Phone    : %s\n", p[pos].phone);
}
}

void updatePatient() {
    int id, pos;

    printf("\nEnter Patient ID to update: ");
    scanf("%d", &id);

    pos = findPatient(id);

    if (pos == -1) {
        printf("\nPatient not found.\n");
        return;
    }

    printf("Enter New Name: ");
    scanf(" %[^\\n]", p[pos].name);

    printf("Enter New Age: ");
    scanf("%d", &p[pos].age);

    printf("Enter New Gender: ");
    scanf("%s", p[pos].gender);

    printf("Enter New Disease: ");
    scanf(" %[^\\n]", p[pos].disease);

    printf("Enter New Phone Number: ");
    scanf("%s", p[pos].phone);

    printf("\nPatient updated successfully!\n");
}

void deletePatient() {
    int id, pos, i;

    printf("\nEnter Patient ID to delete: ");
    scanf("%d", &id);

    pos = findPatient(id);

    if (pos == -1) {
        printf("\nPatient not found.\n");
        return;
    }

    for (i = pos; i < count - 1; i++) {
        p[i] = p[i + 1];
    }

    count--;
    printf("\nPatient deleted successfully!\n");
}

int main() {
    int choice;

    do {
        printf("\n===== HOSPITAL MANAGEMENT SYSTEM =====\n");
        printf("1. Add Patient\n");
        printf("2. Display Patients\n");
        printf("3. Search Patient\n");
        printf("4. Update Patient\n");
        printf("5. Delete Patient\n");
        printf("6. Exit\n");
        printf("Enter your choice: ");

```

```
scanf("%d", &choice);

switch (choice) {
    case 1: addPatient(); break;
    case 2: displayPatients(); break;
    case 3: searchPatient(); break;
    case 4: updatePatient(); break;
    case 5: deletePatient(); break;
    case 6: printf("\nThank you!\n"); break;
    default: printf("\nInvalid choice!\n");
}
} while (choice != 6);

return 0;
}
```

5. SAMPLE OUTPUT

The following is a sample execution of the program:

```
===== HOSPITAL MANAGEMENT SYSTEM =====
1. Add Patient
2. Display Patients
3. Search Patient
4. Update Patient
5. Delete Patient
6. Exit
Enter your choice: 1

Enter Patient ID: 101
Enter Name: Ravi
Enter Age: 35
Enter Gender: Male
Enter Disease: Fever
Enter Phone Number: 9876543210

Patient added successfully!

Enter your choice: 2

----- PATIENT RECORDS -----
ID      Name      Age      Gender  Disease  Phone
-----
101     Ravi         35       Male    Fever    9876543210

Enter your choice: 3
Enter Patient ID to search: 101

Patient Found
ID      : 101
Name    : Ravi
Age     : 35
Gender  : Male
Disease : Fever
Phone   : 9876543210

Enter your choice: 6
Thank you!
```

6. CONCLUSION

The Hospital Management System successfully demonstrates how C programming can be used to organize and manage basic patient information. The menu-driven approach makes the application simple to operate, while structures and functions provide an organized program design.

This project helped in understanding important C programming concepts such as structures, arrays, functions, searching, updating, deleting and switch-case based menu handling.

7. FUTURE ENHANCEMENTS

The system can be improved by adding permanent file storage or a database, doctor and staff management, appointment scheduling, billing, medicine inventory, login authentication and a graphical or web-based interface.

8. REFERENCES

- E. Balagurusamy, Programming in ANSI C.
- The C Programming Language – basic concepts of C programming.
- Class notes and laboratory materials on C programming.